

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Source Code

The complete source code for this experiment is available on GitHub:

https://github.com/srinithi0406/Deep_Learning/tree/main/DL_1

8. Experimental Results

The Single Layer Perceptron was successfully implemented from scratch and trained on the Banknote Authentication dataset using the perceptron learning algorithm. The model was trained for 50 epochs with a learning rate of 0.01. During training number of misclassified samples reduced a lot, indicating that the perceptron was able to learn a suitable decision boundary for the given dataset.

8.1 Dataset Preview

Variance	Skewness	Curtosis	Entropy	Class
3.62160	8.6661	-2.8073	-0.44699	0
4.54590	8.1674	-2.4586	-1.46210	0
3.86600	-2.6383	1.9242	0.10645	0
3.45660	9.5228	-4.0112	-3.59440	0
0.32924	-4.4552	4.5718	-0.98880	0

The first five samples of the Banknote Authentication dataset were displayed successfully after loading the dataset.

8.2 Dataset Information

Dataset Shape	(1372, 5)
Number of Features	4
Target Variable	1
Feature Type	float64
Target Type	int64
Missing Values	None

8.3 Training and Testing Split

Training Samples	1097
Testing Samples	275
Training Labels	1097
Testing Labels	275

The dataset was divided into training and testing sets using an 80:20 split and was normalised using Min-Max Scaling.

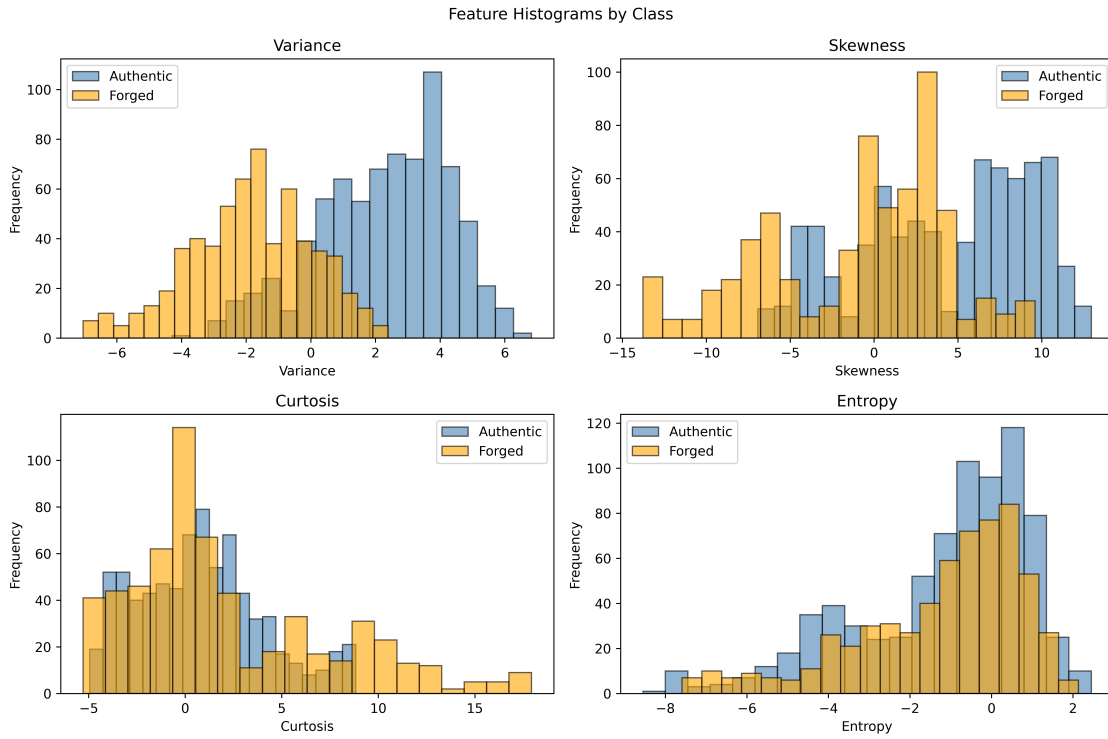
8.4 Initial Training Progress

The number of misclassified samples reduced significantly during initial epochs, showing that perceptron learned from training data through repeated weight and bias updates.

Epoch	Misclassified Samples	Bias
1	168	0.08
2	82	0.10
3	65	0.11
4	49	0.12
5	41	0.13

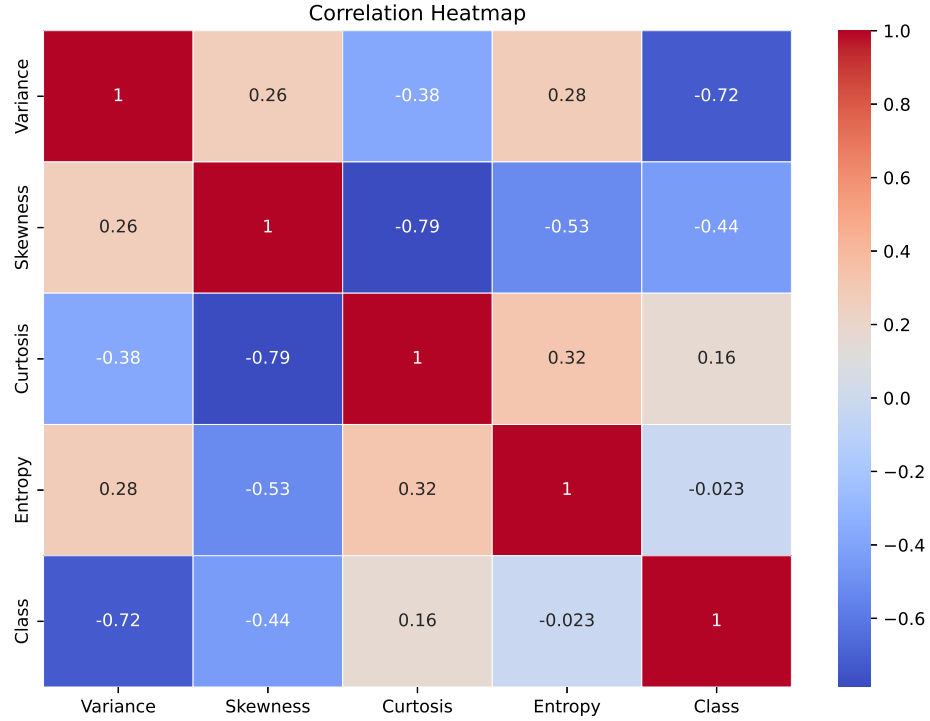
9. Generated Plots with Interpretation

1. Feature Histograms



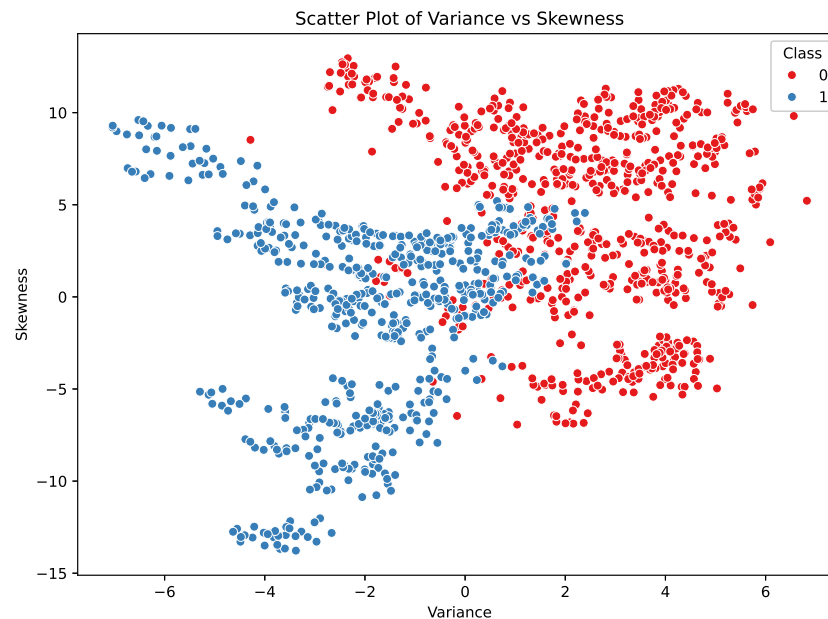
Purpose: Histograms were plotted to understand the distribution and skewness of each numerical feature in the dataset before training our perceptron model. **Interpretation:** The four features have different distributions. Variance and Skewness are spread over a wider range of values. Kurtosis has a longer right tail, showing positive skewness, while Entropy has a longer left tail, showing negative skewness. Most of the values are within main range of each features only, and there are only few extreme values at the ends of the distribution. As each feature has a different distribution, they provide different information about the banknotes. Hence, all four features were used for training the perceptron model.

2. Correlation Heatmap



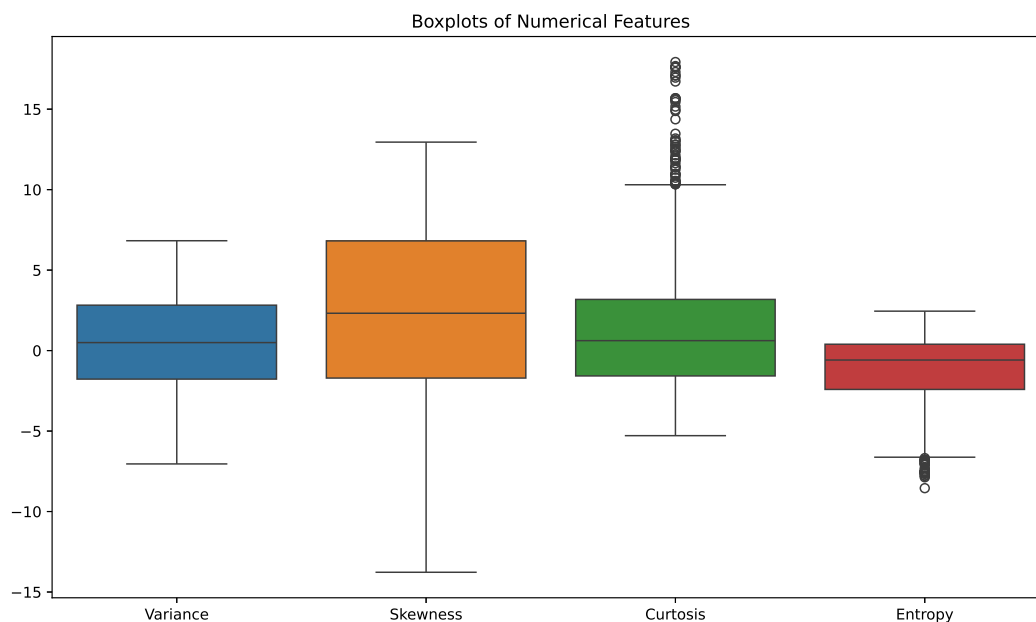
Purpose: To study the relationship between the input features and the target class, a correlation heatmap was generated. **Interpretation:** The heatmap shows that some features have positive correlation while others have negative correlation. Variance has a strong negative correlation with the target class, indicating that it is useful for distinguishing between authentic and forged banknotes. Some pairs of feature are also moderately correlated with each other, but none of them are completely identical. Hence, all the four features are used for training the model.

3. Scatter Plot (Variance vs Skewness)



Purpose: Scatter plot was generated to see how two classes are distributed and also to check whether they can be separated using a straight line. **Interpretation:** The scatter plot showed that most of the authentic and forged banknotes form separate groups when they are plotted using Variance and Skewness. Though few samples overlap in the middle, most of the data can be separated using a straight line. Therefore the dataset is approximately linearly separable using these two features.

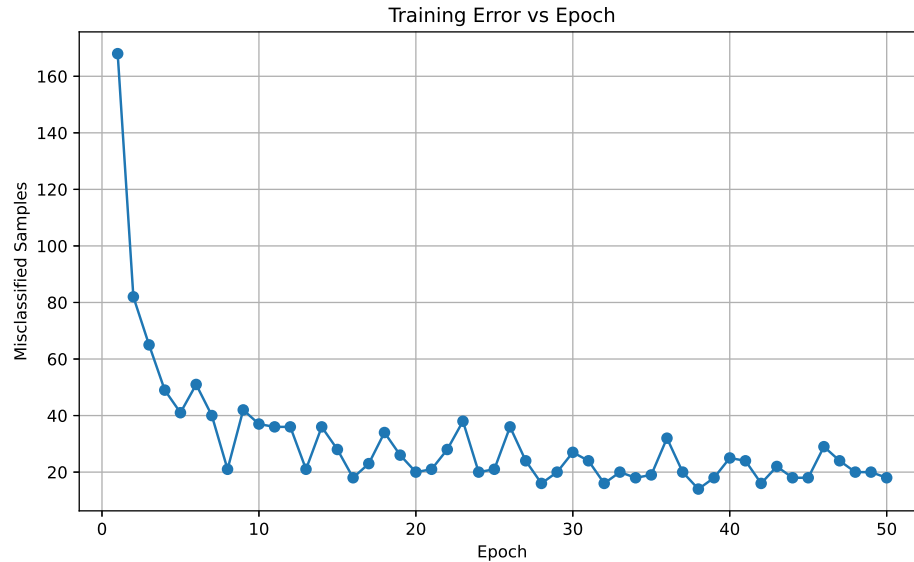
4. Boxplots of Numerical Features



Purpose: Boxplots were plotted to understand how values of each feature are spread. They also help to identify whether any feature contains outliers that are different from the rest of the data.

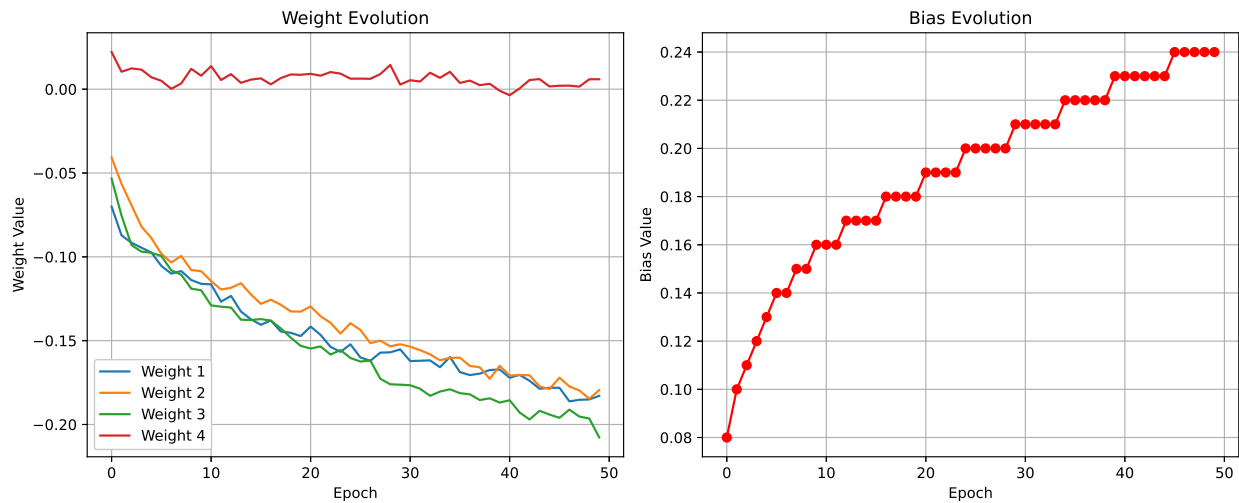
Interpretation: The boxplots showed that all four features have different ranges and different spreads. We can observe that few outliers are present in the data, especially in the Kurtosis feature, where few values lie above the upper whisker. A few outliers are also present in the Entropy feature below the lower whisker. Most of the data lies within the whiskers, showing only a small number of observations are extreme.

5. Training Error vs Epoch



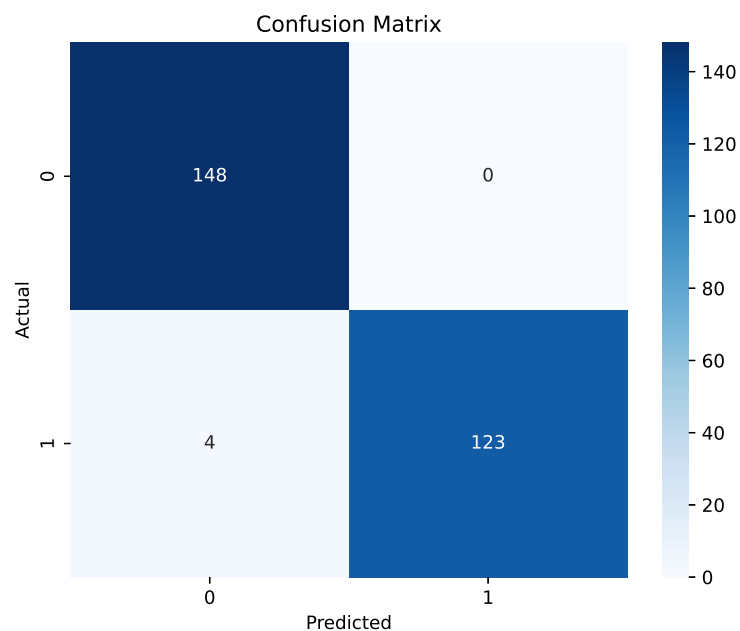
Purpose: This graph was plotted to observe how the training error changes during each epoch and to understand whether the learning process is becoming stable. **Interpretation:** The graph shows that training error is highest during the first epoch with 168 misclassified samples. In the next few epochs, the error decreases quickly, showing that the perceptron is learning the pattern in the training data. After that, the error changes only by a small amount and stays within a lower range. Though there are few small fluctuations between the epochs, overall training error has become much lower than in the beginning. This shows that perceptron has learned a suitable decision boundary.

6. Weight and Bias Evolution



Purpose: This graph was plotted to observe how the weights and bias change during each epoch. helps us understand how the perceptron updates its parameters while learning from the training data. **Interpretation:** The graph shows that weights are updated continuously during training. Weight 1, Weight 2 and Weight 3 decrease slowly during the initial epochs and later become almost stable. Weight 4 changes only by a small amount throughout the training process. The bias increases gradually from 0.08 to 0.24 and finally becomes constant. These show that the perceptron is learning from the training data and the parameter updates are becoming smaller as the learning process approaches convergence.

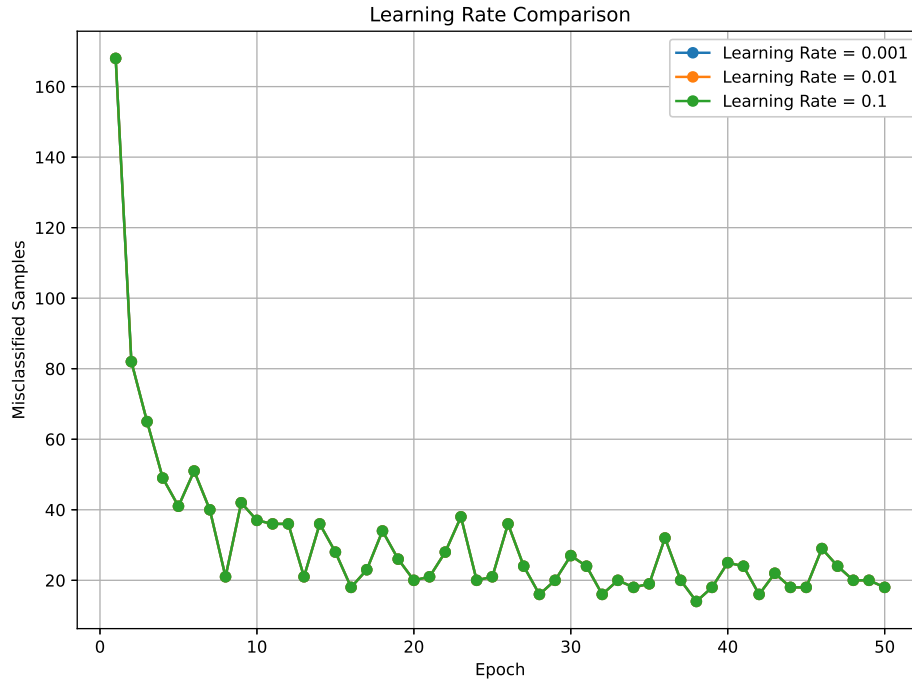
7. Confusion Matrix



True Positives (TP)	123
True Negatives (TN)	148
False Positives (FP)	0
False Negatives (FN)	4

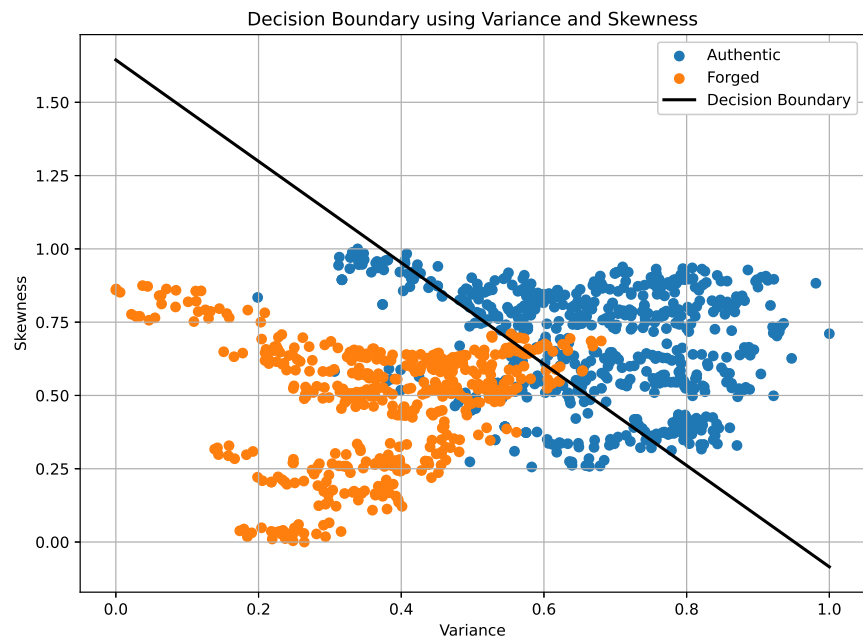
Purpose: The confusion matrix was used to evaluate how well the perceptron classified the banknotes. It shows the number of correct and incorrect predictions made by the model. **Interpretation:** The model correctly classified all 148 authentic banknotes and correctly identified 123 forged banknotes. Only 4 forged banknotes were incorrectly classified as authentic, while there were no false positives. This results in high accuracy, perfect precision, and a high F1-score, showing that the perceptron performed well on this binary classification task.

8. Learning Rate Comparison



Purpose: This graph was plotted to compare how different learning rates affect the learning behaviour of the perceptron during training. **Interpretation:** The graph shows that the curves for all three learning rates almost completely overlap. This is because the perceptron uses the Step Activation Function, where the prediction depends only on whether the weighted sum is greater than or less than zero. Changing the learning rate only changes the size of the weight update and not the prediction itself. Therefore, the number of misclassified samples remains almost the same for all three learning rates, resulting in overlapping curves. In this experiment, changing the learning rate did not produce any noticeable difference in the learning behaviour of the perceptron.

9. Decision Boundary(for variance and skewness)



Inference: This plot shows the decision boundary learned by the perceptron using only the Variance and Skewness features. The boundary is a straight line that separates a large number of authentic and forged banknotes. However, some samples, especially of the authentic notes are present on the wrong side of the line because these two features alone cannot completely separate all the data. This shows that the perceptron has learned the best possible linear boundary for these two features, but a few misclassifications still exists.

10. Performance Tables

Training Summary

Dataset Size	1372 Samples
Train/Test Split	1097 / 275 (80% : 20%)
Learning Rate	0.01
Epochs	50
Final Weights	$[-0.16966474, -0.16589198, -0.18542884, 0.00239884]$
Final Bias	0.22
Accuracy	98.55%
Precision	100.00%
Recall	96.85%
F1-score	98.40%

Epoch-wise Learning

Epoch	Errors	W1	W2	W3	W4	Bias
1	168	-0.0701	-0.0408	-0.0533	0.0221	0.08
5	41	-0.0974	-0.0888	-0.0976	0.0071	0.13
10	37	-0.1161	-0.1085	-0.1199	0.0080	0.16
15	28	-0.1373	-0.1225	-0.1377	0.0057	0.17
20	20	-0.1473	-0.1327	-0.1531	0.0086	0.18
25	21	-0.1522	-0.1396	-0.1604	0.0063	0.20
30	27	-0.1552	-0.1521	-0.1763	0.0027	0.21
35	19	-0.1598	-0.1602	-0.1790	0.0104	0.22
40	25	-0.1672	-0.1650	-0.1869	-0.0009	0.23
45	18	-0.1781	-0.1789	-0.1941	0.0016	0.23
50	18	-0.1829	-0.1795	-0.2078	0.0059	0.24

11. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.

The Step Activation Function gives only two outputs, either 0 or 1. If the input is greater than or equal to the threshold, it returns 1. Otherwise it returns 0. This makes it suitable for simple binary classification problems such as the perceptron, whereas the sigmoid Activation Function gives a value between 0 and 1 instead of only two values. The output changes smoothly as the input changes, so it can also be interpreted as a probability. The Step Activation Function is not used in modern deep learning because it is not differentiable. Since its output changes suddenly, gradients cannot be calculated during backpropagation. Modern neural networks learn by using gradient-based optimization, so activation functions such as Sigmoid, ReLU and Tanh are preferred. The Step function is mainly used for understanding the basic working of a perceptron.

2. Compare the implementation with Scikit-learn's Perceptron.

The custom perceptron achieved an accuracy of **98.55%**, while the Scikit-learn perceptron achieved **97.45%**. Both models obtained a precision of 100%, but the custom implementation achieved a slightly higher recall and F1-score. Though the learned weights and bias are slightly different, both models produced similar results.

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.

The experiment was repeated using learning rates of **0.001**, **0.01**, and **0.1**. As shown in the learning rate comparison plot, all three learning rates produced almost identical training error curves. This indicates that changing the learning rate had very little effect on the convergence in this experiment because the perceptron uses the Step Activation Function.

4. Explain why a Single Layer Perceptron cannot solve the XOR problem.

A Single Layer Perceptron can separate data only using one straight line. In the XOR problem, the points belonging to the same output classes are present on opposite sides, so a single straight line cannot separate the two classes correctly. Because of this, the perceptron cannot classify all

the inputs correctly. To solve the XOR problem, more than one layer is needed to learn a better separation.

5. Study the effect of feature normalization on model convergence.

Feature normalization was applied using Min-Max Scaling before training the perceptron. This converted all the feature values to the same range 0 to 1. Hence, the model learned more smoothly and the training errors reduced quickly during the initial epochs before becoming stable. This shows that normalization helped the perceptron converge faster and made the learning process more stable.

Additional Programming Task: Perceptron Learning for Logic Gates

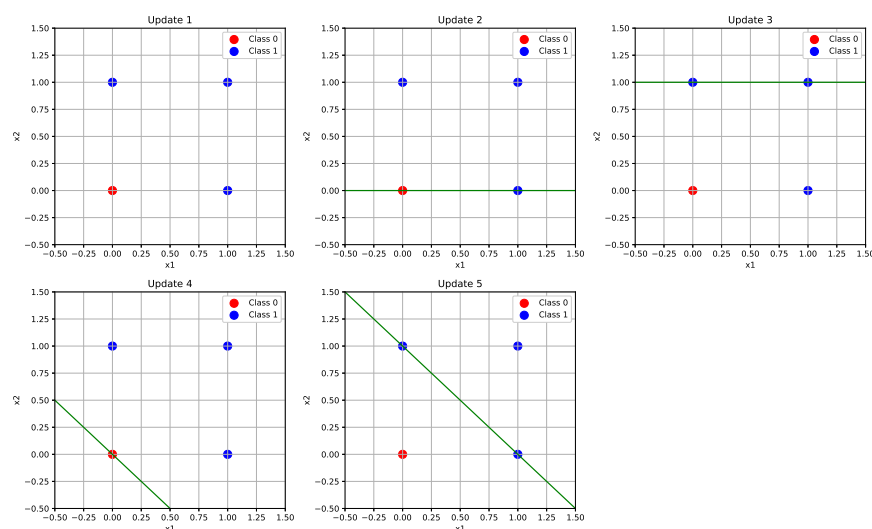
Implemented perceptron learning algorithm from scratch for OR, AND and NOT gates. For each gate, updated the weights and bias whenever misclassification occurred. The decision boundary corresponding to every weight update was plotted to visualize how the perceptron slowly learned the correct linear separation.

OR Gate

Weight and Bias Updates

Update	Weights	Bias
1	[0.0, 0.0]	-0.1
2	[0.0, 0.1]	0.0
3	[0.0, 0.1]	-0.1
4	[0.1, 0.1]	0.0
5	[0.1, 0.1]	-0.1
Final	[0.1, 0.1]	-0.1

OR Gate Decision Boundary Evolution



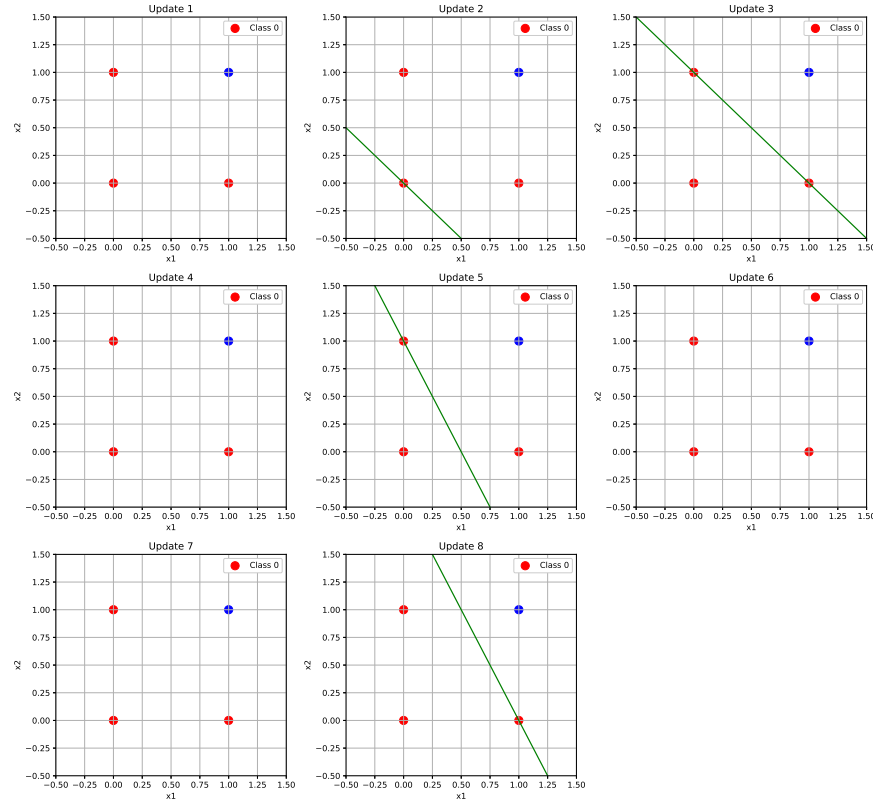
Inference: After four epochs the OR gate converged. After every weight update the decision boundary shifted and finally separated the single negative sample from the three positive samples. The final weights remained stable, indicating successful convergence.

AND Gate

Weight and Bias Updates

Update	Weights	Bias
1	[0.0, 0.0]	-0.1
2	[0.1, 0.1]	0.0
3	[0.1, 0.1]	-0.1
4	[0.1, 0.0]	-0.2
5	[0.2, 0.1]	-0.1
6	[0.2, 0.0]	-0.2
7	[0.1, 0.0]	-0.3
8	[0.2, 0.1]	-0.2
Final	[0.2, 0.1]	-0.2

AND Gate Decision Boundary Evolution



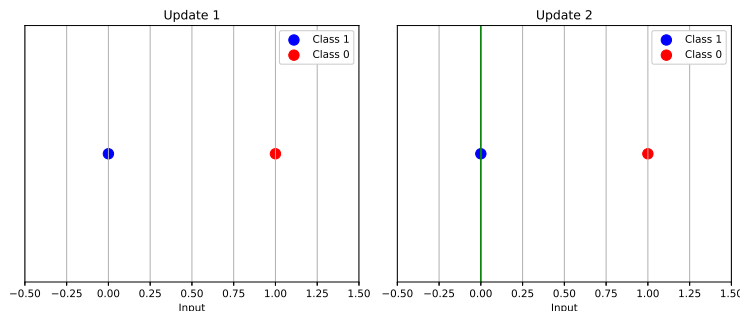
Inference: AND gate required more weight updates than OR gate before converging. The decision boundary slowly shifted until only the input (1,1) was classified as positive, demonstrating successful learning of a linearly separable function.

NOT Gate

Weight and Bias Updates

Update	Weight	Bias
1	-0.1	-0.1
2	-0.1	0.0
Final	-0.1	0.0

NOT Gate Decision Boundary Evolution



Inference: After three epochs, NOT gate converged. As it has only one input feature, final decision boundary corresponds to a single threshold. The perceptron learned the NOT operation correctly after two parameter updates itself.

12. Discussion

In this experiment, we built a Single Layer Perceptron from scratch and trained it using the Banknote Authentication dataset. First, we looked at the dataset using different graphs to understand the data better. Then the dataset was divided into training data and testing data. Before training, the feature values were normalized so that all the inputs were in a similar range. This helped the model learn more smoothly.

The perceptron used the Step Activation Function to make predictions. For every training sample, the weighted sum of the input features and bias was calculated. If the value was greater than or equal to zero, the output was predicted as 1, otherwise it was predicted as 0. Since this is a binary classification problem, the Step Activation Function was a simple and suitable choice.

Whenever the prediction was wrong, the weights and bias were updated. During the first few epochs, the model made many wrong predictions, but as the training continued, the number of wrong predictions became smaller. The weights and bias also became more stable, which showed that the perceptron was learning from the training data.

After training, the final weights and bias were used to predict the classes of the test data. These predictions were compared with the actual class labels to calculate the accuracy, precision, recall and F1-score. The model achieved an accuracy of about 98.5%, which means it predicted almost all the banknotes correctly. The confusion matrix also showed that only a few samples were predicted wrongly.

The results show that a Single Layer Perceptron works well for this dataset because most of the banknotes can be separated using a single decision boundary. Overall, this experiment helped us understand how the Step Activation Function, weight updates and multiple training epochs work together to improve the model and make accurate predictions.

13. Conclusion

In this experiment, we successfully achieved the objectives of implementing a Single Layer Perceptron from scratch and using it for binary classification. We understood how an artificial neuron works, how the perceptron learning algorithm updates the weights and bias, and how the Step Activation Function is used to make predictions. We also visualized the learning process using different graphs and evaluated the model using performance metrics. Overall, this experiment gave us a clear understanding of how a Single Layer Perceptron works and how it can be used to solve a real-world binary classification problem.

14. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.